parentheses are left empty. Finally, we have the first of two curly braces that enclose the code needed in the function.

Any new function that we add to our sketch must be done outside any other function that we might have, including the `setup()` and `loop()` functions that every sketch using the Arduino libraries will have. For the most part, it really doesn't mater where a function is declared within the sketch, although we often find ourselves usually declaring them after the `loop()` function, more out of habit than anything.

## Calling Functions

With a function written, such as our `blinkLED()` function from earlier, we need to **call** the function when we want it to do its job. The thing is, whenever we have talked about a function in this book, we have been calling it in our code. So to call our function, we would write a simple statement like the following:

```
blinkLED();
```

And that is all there is to it. When the function is called, program flow jumps to that function, executes the block of code, and when the function has ended, program flow returns to the next line after our function call. So, building on our new function, if in our `loop()` function we only had the following:

```
void loop() {
  blinkLED();
}
```

This would effectively rewrite our blinking LED sketch by using a function. Each time through the loop, the `blinkLED()` function is called, and inside that function, the LED on pin 13 is turned on for 1 second and turned off for 1 second. When all of the statements inside the function have been executed, program flow returns to the `loop()` function and it all starts over again.

## Function Returns

Our example function `blinkLED()` doesn't admittedly do all that much. To make things more interesting, we could ask our function to perform some sort of operation and give us a result back in a nice value that we could then do something with. This act of calling a function and passing a value from that function back to its calling function is known as a **function return**.

In declaring a function, the first thing we need to declare is the function's return data type. For the most part in this book, when a function has been declared, it has been done using the void data type. I know, void wasn't one of the options when we looked at variable data types. That's because it's not really that useful for variables, but when it comes to functions, it's a good idea to tell the compiler that the function being declared will have no value. In order for our function to have a value though, we need to declare a data type that matches the expected value to be returned. Take the following, for example:

```
int readSensor() {
```

Here we are declaring a new function called `readSensor()` of the `int`, or integer data type. Just as with integer type variables, this will give us an expected range of values from -32,768 to 32,767. We can pretty much use any of the data types discussed earlier for variables, including long, unsigned, or even boolean.

Now let's look at how the function can actually return a value. Let's say that we want our `readSensor()` function to not only read a sensor, but to also smooth out our sensor readings by taking 5 samples very quickly, averaging those samples, and returning that value in a format that we can use for PWM. The following is the function: